

VisionEngine

Воспринимайте интерфейс как пользователь — компьютерное зрение плюс LLM-зрение для анализа и навигации.

Краткое описание

VisionEngine — это независимый инструмент Go, сочетающий классическое компьютерное зрение с LLM-зрением для анализа пользовательских интерфейсов, выявления элементов UI и визуальных дефектов, а также построения графов переходов между экранами приложения. Поддерживает подключаемые бэкенды для различных провайдеров зрения, а OpenCV доступен только при сборке с соответствующим тегом.

Описание

Модуль Go для анализа интерфейсов и построения навигационных графов. Включает слой анализатора (элементы UI, сравнение экранов, визуальные дефекты), навигационный граф с поиском пути методом BFS и экспортом в форматах DOT, JSON и Mermaid, а также адаптеры LLM-зрения для GPT-4o, Claude, Gemini, Qwen-VL и других.

Подробное описание

Большинство инструментов автоматизации тестирования интерфейсов фактически слепы. Они полагаются на деревья доступности и DOM-селекторы — машинное представление интерфейса — и упускают всё, что видит человек: отображается ли кнопка, не сломалась ли вёрстка, тот ли экран открылся, который ожидался. VisionEngine устраняет этот пробел, наделяя автоматизацию настоящим восприятием — способностью видеть интерфейс и анализировать его так,

как это делает человек. Инструмент структурирован в четыре взаимодействующих слоя, которые последовательно преобразуют пиксели в понимание приложения в целом.

Анализатор определяет стабильный контракт — интерфейсы (Analyzer, VideoProcessor) и типы данных (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) с эталонной реализацией StubAnalyzer. Это позволяет пользователям обнаруживать элементы, сравнивать экраны и выявлять визуальные дефекты в рамках неизменного контракта.

Навигационный граф расширяет анализ с отдельного экрана на всё приложение, моделируя его как направленный граф переходов между экранами с поиском пути методом BFS и тремя форматами экспорта (DOT, JSON, Mermaid). Это позволяет автоматизации не только видеть экран, но и планировать маршрут к любому другому. В комплекте поставляются тестовые наборы для нагрузочного, автоматического, интеграционного и security-тестирования.

Слой **LLM-зрения** добавляет современные возможности мультимодального анализа: интерфейс VisionProvider с адаптерами для OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica и Ollama. Провайдеры объединяются в цепочку отката (FallbackChain), благодаря чему сбой, ограничение по запросам или слабый провайдер плавно передаёт управление следующему, не нарушая работу всего процесса.

Слой **конфигурации** отвечает за загрузку и валидацию переменных окружения, а все сообщения об ошибках для пользователя проходят через i18n.Translator.

Ключевое решение, делающее этот инструмент по-настоящему применимым на практике, — опциональность тяжёлых зависимостей. Привязки OpenCV доступны только при сборке с тегом -tags vision, а в стандартной сборке используются заглушки. Это позволяет модулю компилироваться, тестироваться и работать на любой хост-системе с Go 1.25+ без установки OpenCV. Тяжёлые зависимости подключаются только при явном выборе пользователя. Именно это позволяет VisionEngine запускаться в обычном CI-раннере без специального образа.

Полностью независимый в соответствии с конституцией (CONST-051(B)), он подключается потребителями — в частности, HelixQA — как равноправный подмодуль кодовой базы, предоставляя UI-тестированию на основе данных настоящие «глаза».

Почему мы его создали

Автоматизация тестирования интерфейсов, основанная только на деревьях доступности или селекторах, упускает из виду то, что видит пользователь. VisionEngine добавляет реальное визуальное понимание — обнаружение элементов, сравнение экранов и рассуждения на основе LLM-зрения, — а также навигационную карту экранов приложения, чтобы автоматизация могла не только воспринимать интерфейс, но и прокладывать путь через него.

Почему это меняет правила игры

Решение объединяет два обычно несовместимых подхода — быструю, детерминированную классическую компьютерную обработку изображений и гибкое, семантическое LLM-зрение — за единым интерфейсом с цепочкой откатов. Пользователь получает точность одного подхода и логику другого, не выбирая между ними. А благодаря тому, что OpenCV остаётся строго опциональным, исчезает обычный налог на эту мощь: любой проект на Go может обрести реальное восприятие интерфейса, не таща в сборку нативный инструментарий компьютерного зрения.

Что нового

- Двойное восприятие: классическое компьютерное зрение (OpenCV/GoCV) плюс мультипровайдерное LLM-зрение с цепочкой откатов.
- Навигационный граф с поиском пути в ширину и экспортом в DOT/JSON/Mermaid.
- Управление сборкой OpenCV через теги, чтобы модуль оставался собираемым и тестируемым без нативных зависимостей.
- Полностью развязанный, интернационализированный submodule с единой кодовой базой (используется в HelixQA).

Проблемы и решения

- **Сложности с тяжёлыми нативными зависимостями:** решены с помощью условной сборки через `-tags vision` и заглушек по умолчанию, чтобы CI и хосты без OpenCV могли собирать и тестировать проект.
- **Ненадёжность провайдеров компьютерного зрения:** решена через интерфейс `VisionProvider` и композицию `FallbackChain`.

- **Картирование сложных потоков приложения:** решено с помощью направленного навигационного графа, поиска пути в ширину и экспорта в нескольких форматах.
- **Связанность:** устранена через CONST-051(B) и слой интернационализации.

Технологический стек (зачем и как)

- **Go (1.25+)** — ядро модуля и все четыре слоя.
- **GoCV / OpenCV** — классическое компьютерное зрение, управляемое тегами сборки.
- **Провайдеры LLM-зрения (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — мультимодальное понимание интерфейсов через адаптеры.
- **Алгоритмы графов (поиск в ширину)** — навигация и прокладывание пути.
- **Экспортеры DOT / JSON / Mermaid** — визуализация навигационного графа.
- **Переводчик i18n** — развязанные пользовательские строки.